

LINKSFOUNDATION.COM

Applied Data Science Project

L7 – Large Language Models



**Politecnico
di Torino**



e l i s
European Laboratory for Learning and Intelligent Systems

LLMs = foundation models; numerous available

LLMs = Large language models. An LLM is a foundation model

The term LLM appeared later on,
anyway BERT can be considered
a canonical example

Free / Open Access

BERT (2018)

GPT-2 (OpenAI, 2019)
GPT-Neo / GPT-J / GPT-NeoX
(EleutherAI, 2020)
BLOOM (BigScience, 2022)
Falcon (TII, 2023)
Mistral (Mistral AI, 2023)
Gemma (Google, 2024)
OLMo (AI2, 2024)

Open Models

Closed Models

Few small demos/free tiers (GPT-3.5,
Claude, Gemini) with limited usage

Commercial / API

LLaMA 2 & 3 (Meta, 2023–24) have open
weights but restricted license
Some Mistral commercial APIs

GPT-4 / GPT-4o / GTP-5 (OpenAI)
Claude (Anthropic)
Gemini (Google DeepMind)
Cohere Command-R+
Aleph Alpha (Luminous)
xAI's Grok

Are ChatGPT or Claude LLMs?

No, they are applications that wrap LLMs

Because they also offer QA support, routing on other foundation models (for instance image generation), they profile users, and they learn from user experiences, while offering a web page for logging and use



LLMs

LLMs are state-of-the-art systems across many language-related tasks, showing remarkable fluency in text generation and strong few-shot learning abilities. At the same time, they remain difficult to interpret and raise important ethical and scalability concerns

LLMs are trained on vast amounts of text data to understand, generate, and analyse human language. They utilize deep learning and transformer neural networks with self-attention mechanisms to process and contextualize information, enabling them to perform tasks like translation, summarization, question answering, and content creation.

An LLM follows the same peculiarities seen for foundation models: large and clean data, large computing resources for the making. They are the best choice when dealing with a task that processes or produces text

They are large

Model

GPT-4 / GPT-4o (OpenAI)
Gemini (Google DeepMind)
Claude (Anthropic)
BLOOM (BigScience, 2022)
LLaMA 2 / 3 (Meta)
Aleph Alpha – Luminous
Mixtral 8×7B (Mistral, 2023)
Falcon (TII, 2023)
Cohere Command-R+
GPT-NeoX (EleutherAI)
OLMo (AI2, 2024)
Gemma (Google, 2024)
Mistral 7B (2023)
GPT-J (EleutherAI, 2021)
GPT-2 (OpenAI, 2019)
BERT Large (Google, 2018)
BERT Base (Google, 2018)

Max Parameters (weights)

Not disclosed (estimate 100B–1T+)
Not disclosed (estimate 100B–1T+)
Not disclosed (estimate 50–100B+)
176B
70B
~70B
46.7B active (MoE)
40B
Tens of B (undisclosed)
20B
7B (larger planned)
2B & 7B
7B
6B
1.5B
340M
110M

Pattern from 2018 onwards

Numerous models been released

Common patterns the increase in the number of parameters used for storing the weights of the neural model

To cope with the increase in number of parameters, a new discipline took place: model optimisation, ie using the large model and reducing the number of parameters, while preserving good (or nearly the same) performance of the original models



The origins of Language Models

Once upon a time

In 1948, Shannon introduced the concept of language as a stochastic process

Language is modelled as a sequence of words with probabilities

Early experiments were done in applying statistics to compute “how good is a group of world”



Language Model

It is the probability distribution over sequences of tokens

Suppose we have a vocabulary V of a set of tokens. A language model p assigns each sequence of tokens $w_1, \dots, w_t \in V$ a probability (a number between 0 and 1):

$$p(w_1, \dots, w_t)$$

The probability intuitively tells how “good” a sequence of tokens is (one probability to the whole sentence)

For example, if the vocabulary is $V = \{taken, at, ADSP, course, is, the, Polito\}$, the language model may assign:

$$p(the, ADSP, course, is, taken, at, Polito) = 0.1,$$

$$p(the, Polito, course, is, taken, at) = 0.01,$$

$$p(course, the, ADSP, is, Polito, taken, at) = 0.001.$$

Markov models

From 1960s till 1990s

Early practical approach to language modeling.

Markov assumption: the probability of the next word depends only on the previous $n-1$ words.
Implemented as **n-gram models** (e.g., bigram, trigram).
Trained by counting frequencies in text corpora.

Trade-off: larger $n \rightarrow$ better context, but more data required.

Formulation:

General probability of a sequence:

$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^T P(w_t | w_1, \dots, w_{t-1})$$

Markov (n-gram) assumption:

$$P(w_t | w_1, \dots, w_{t-1}) \approx P(w_t | w_{t-n+1}, \dots, w_{t-1})$$

Example (bigram):

$$P(w_1, w_2, w_3) \approx P(w_1) P(w_2 | w_1) P(w_3 | w_2)$$

Autoregressive model

From 2000s onwards

Factorizes sequence probability into a product of conditional probabilities

Unlike n-grams, the model conditions on the entire history of previous tokens

$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^T P(w_t | w_1, \dots, w_{t-1})$$

The formulation is identical, but truncations. For example:

Given a $V=(the, ADSP, course, is, taken, at, Polito)$

$P(the, ADSP, course, is, taken, at, Polito) = P(the) *$

$P(ADSP|the) *$

$P(course|the, ADSP) *$

$P(is|the, ADSP, course) *$

$P(taken|the, ADSP, course, is) *$

$P(at|the, ADSP, course, is, taken) *$

$P(Polito|the, ADSP, course, is, taken, at)$

Autoregressive model

From 2000s onwards

Assume we have the following $P(\text{the}, \text{ADSP}, \text{course}, \text{is}, \text{taken})$, computing the next word means computing the probability of

$$P(\text{Polito} | \text{the}, \text{ADSP}, \text{course}, \text{is}, \text{taken}, \text{at}) = 0.2$$

$$P(\text{University} | \text{the}, \text{ADSP}, \text{course}, \text{is}, \text{taken}, \text{at}) = 0.15$$

$$P(\text{Torino} | \text{the}, \text{ADSP}, \text{course}, \text{is}, \text{taken}, \text{at}) = 0.1$$

$$P(\boxed{} | \text{the}, \text{ADSP}, \text{course}, \text{is}, \text{taken}, \text{at}) = 0.001$$

$$P(w_t | \text{the}, \text{ADSP}, \text{course}, \text{is}, \text{taken}, \text{at}) = 0.00001$$

Done for the entire catalogue of words

Autoregressive model & neural networks

From 2000s onwards

Tokenize the prefix $\rightarrow (w_1, \dots, w_6)$ where $w_1 = \text{the}, \dots, w_6 = \text{at}$

Encode the whole history with the model $f_\theta(\text{RNN/LSTM/Transformer})$ to get logits $z \in \mathbb{R}^{|V|}$ for the next token:

$$z = f_\theta(w_1, \dots, w_6)$$

Softmax to get probabilities over the vocabulary V :

$$P(w \mid \text{the, ADSP, course, is, taken, at}) = \exp(z_w) / \sum_{v \in V} \exp(z_v)$$

Decode a next token:

- Greedy: $\arg\max_w P(w \mid \cdot)$
- Top-k sampling: sample from the top subset
- Beam search: keep multiple partial hypotheses

Most adopted neural architectures

From 2000s onwards

RNN (1982)

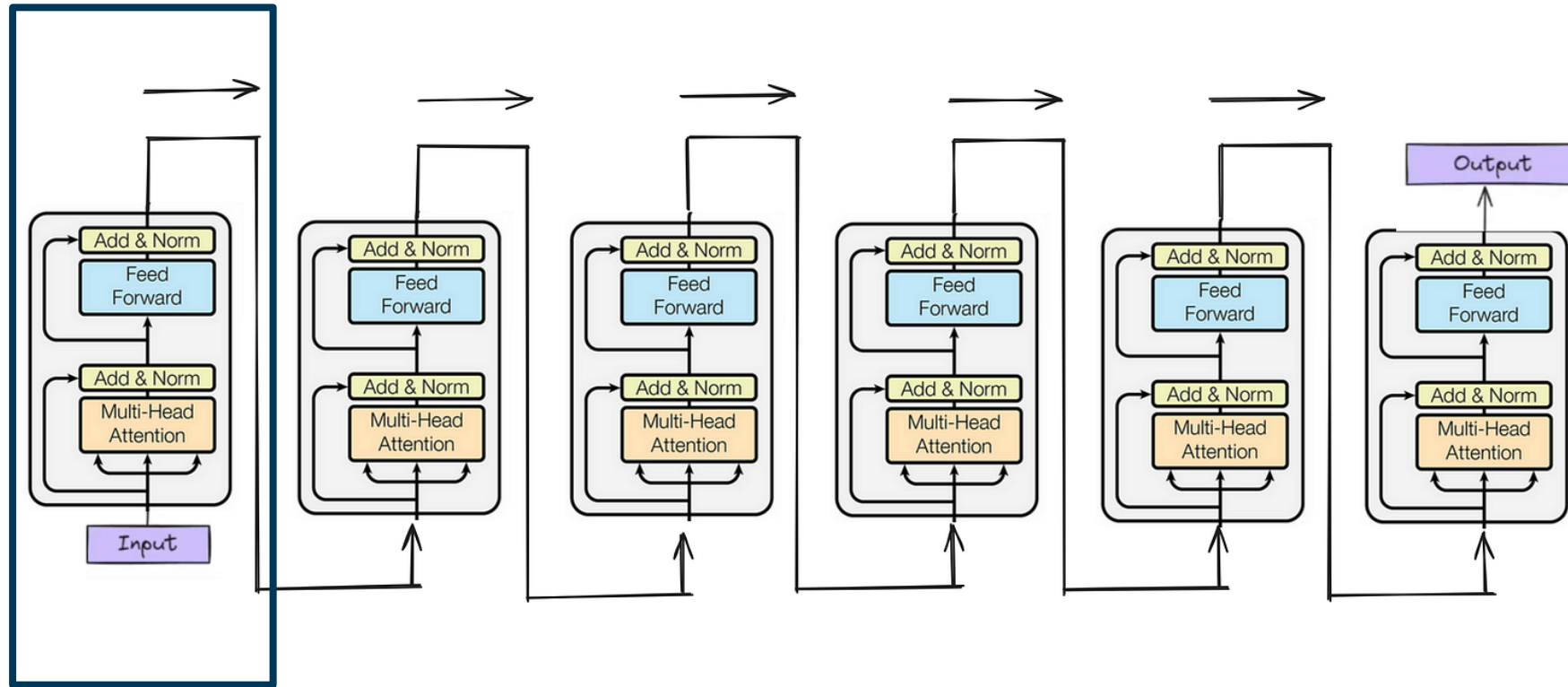
LSTM (1997) – with the variation of GRU (2014)

Transformers (2017)

The three have in common the concept of sequence, however the latter introduced the concept of parallelization of the inputs, thus considering context in processing (ie more tokens altogether)

Transformers-based architecture

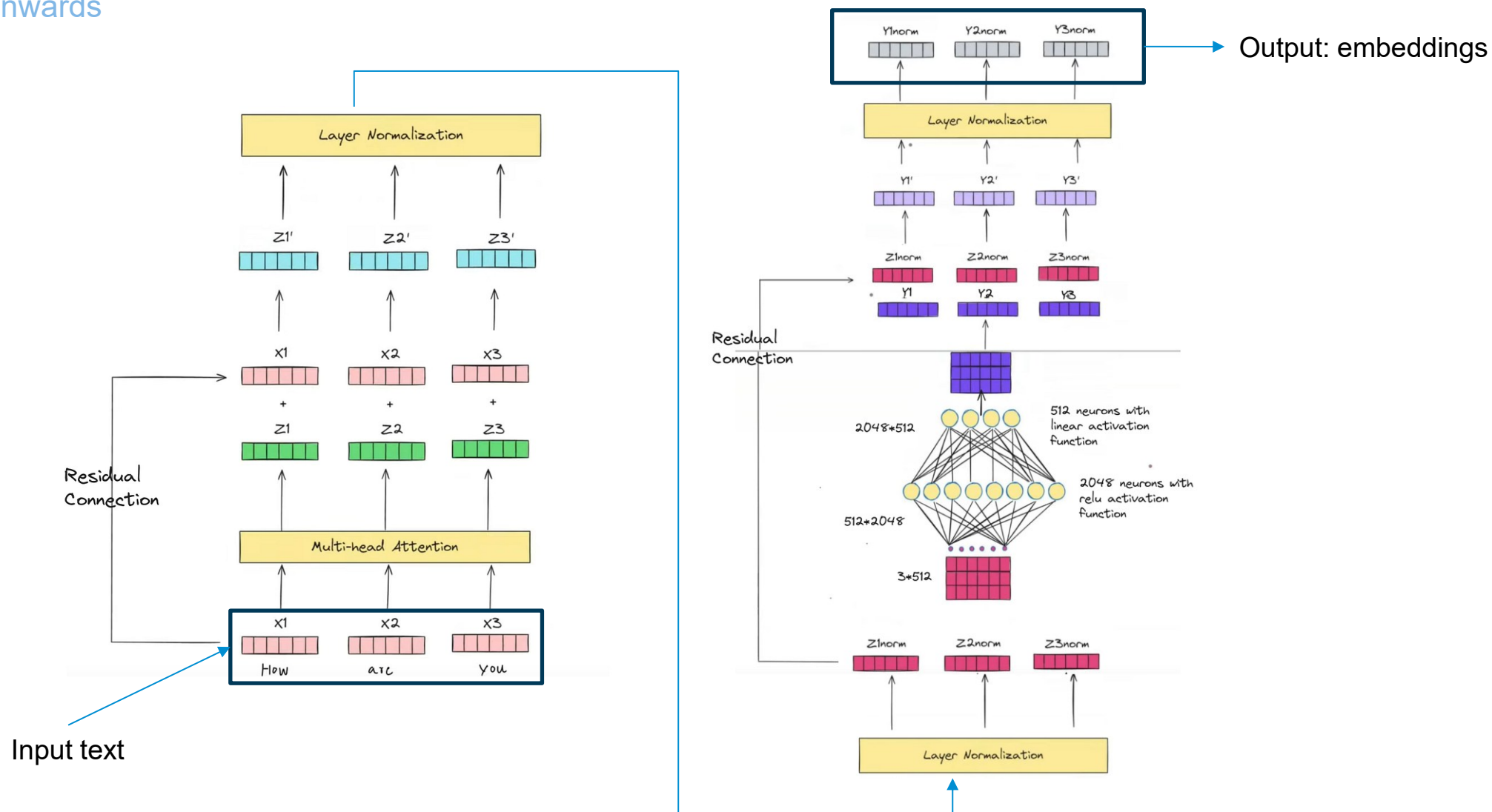
From 2000s onwards



Unrolling of the network

Transformers-based architecture

From 2000s onwards



Hands on

From 2000s onwards

Text similarity with 3 sentences

"The ADSP course is taken at Polito."

"Students attend the ADSP course at Politecnico di Torino."

"Transformers enable large language models."



BERT & Text similarity.ipynb



Thank you for your attention.

Questions?



CONTACTS

Giuseppe Rizzo

Adjunct Professor (Politecnico di Torino)
Program Manager (LINKS Foundation) and

giuseppe.rizzo@polito.it

FONDAZIONE LINKS
Via Pier Carlo Boggio 61 | 10138 Torino
P. +39 011 22 76 150
LINKSFOUNDATION.COM